



UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO
FACULTAD DE CIENCIAS DE LA COMPUTACIÓN Y
DISEÑO DIGITAL
CARRERA DE SOFTWARE

APLICACIONES WEB – SOFT-R – A – [5TO NIVEL] | CORTE 1

TEMA:

GA – Práctica en clase y TA – Tarea fuera de clase
Proyecto ASP.NET Core MVC — GestBiblioteca

ESTUDIANTE:

PANAMA MURILLO MOISES ANTONIO

DOCENTE:

Guerrero Ulloa Gleiston Ciceron

MATERIA:

Aplicaciones Web 5to Nivel

Contents

1	Introduccion	2
2	Entorno y herramientas	2
3	Paso 1 — Creacion del proyecto	2
3.1	Comandos ejecutados	2
3.2	Descripcion	2
3.3	Evidencia en terminal	2
3.4	Estructura de carpetas generada	4
4	Paso 2 — Modelo Prestamo y PrestamoController	4
4.1	Codigo del modelo	4
4.2	Archivo <code>Prestamo.cs</code> en el editor	5
4.3	Codigo del controlador (Index y Create)	5
4.4	Archivo <code>PrestamoController.cs</code> en el editor	7
5	Paso 3 — Vistas Razor	7
5.1	Codigo de la vista <code>Index</code>	7
5.2	Vista <code>Index</code> en el navegador	8
5.3	Codigo de la vista <code>Create</code>	8
5.4	Formulario <code>Create</code> en el navegador	9
6	Paso 4 — Accion Delete con confirmacion, POST y CSRF	10
6.1	Codigo del controlador (Delete)	10
6.2	Vista de confirmacion en el navegador	11
7	Paso 5 — Verificacion de la validacion (server-side y cliente)	11
7.1	Errores de validacion en el formulario	11
7.2	Prestamo creado exitosamente	12
8	Repositorio GitHub	12
8.1	Flujo de trabajo utilizado	12
8.2	Historial de commits	13
8.3	Enlace al repositorio	13
9	Extensión de la práctica — Tarea TA: Módulo de Productos	13
9.1	Paso 1 TA — Creación de la rama y el modelo <code>Producto</code>	13
9.2	Paso 2 TA — <code>ProductoController</code>	14
9.3	Paso 3 TA — Vistas Razor: tabla HTML, Create y Delete	16
9.4	Paso 4 TA — Verificación de validación cliente y servidor	19
9.5	Paso 5 TA — README actualizado	20
9.6	Flujo de Git/GitHub — Rama <code>feature/practica-aspnet</code>	20
9.7	Pull Request y enlace al repositorio	21
10	Conclusiones	22

1 Introduccion

Este proyecto implementa el modulo de **prestamos** del sistema **GestBiblioteca** utilizando **ASP.NET Core MVC**. A traves de la gestion de prestamos de libros se aplicaron los conceptos de validacion de datos con *Data Annotations*, proteccion de formularios con **AntiForgeryToken**, eliminacion segura mediante confirmacion POST con proteccion CSRF, y manejo de rutas mediante controladores y vistas Razor.

2 Entorno y herramientas

- **IDE:** IntelliJ IDEA (terminal integrada para compilar y ejecutar)
- **SDK:** .NET 10
- **Sistema operativo:** Windows
- **Control de versiones:** Git + GitHub
- **Repositorio:** <https://github.com/MoisesPanama/GestBiblioteca>
- **Puerto local:** <http://localhost:5248/Prestamo> y <http://localhost:5248/Producto>
- **Ramas del proyecto:** main, feature/extension-delete-csrf, feature/practica-aspnet

3 Paso 1 — Creacion del proyecto

3.1 Comandos ejecutados

```
mkdir GestBiblioteca
cd GestBiblioteca
dotnet new mvc -n GestBibliotecaApp
cd GestBibliotecaApp
dotnet run
```

3.2 Descripcion

Se ejecuto el comando `dotnet new mvc -n GestBibliotecaApp` en la terminal integrada de IntelliJ IDEA, generando automaticamente la estructura base del proyecto con las carpetas **Controllers**, **Models**, **Views**, **wwwroot** y los archivos **Program.cs** y **GestBibliotecaApp.csproj**. Al ejecutar `dotnet run` el servidor quedo escuchando en <http://localhost:5248>, confirmando que el proyecto se creo sin errores.

3.3 Evidencia en terminal

Se ejecutaron los comandos en la terminal integrada de IntelliJ IDEA, generando la estructura base y confirmando que el servidor levanto correctamente.

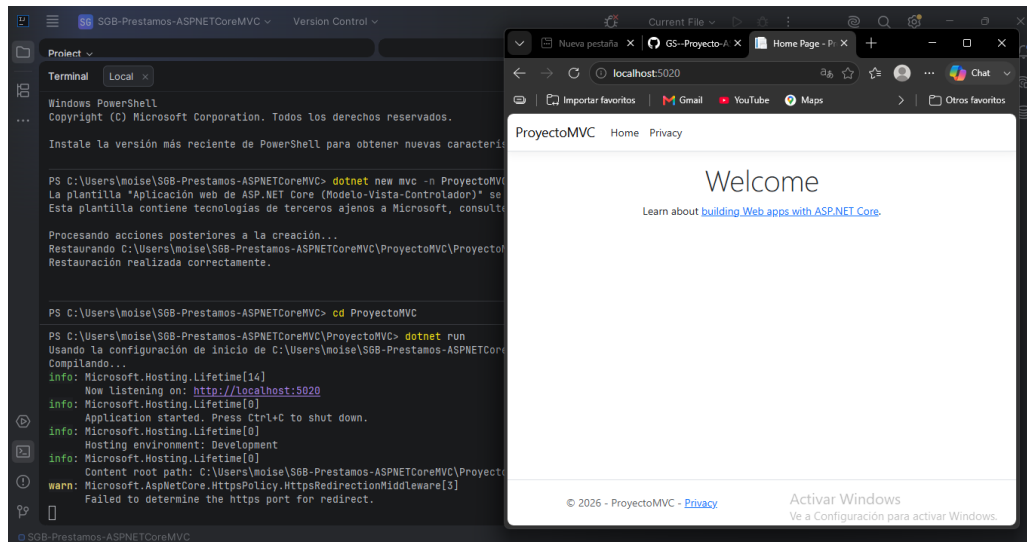


Figure 1: Terminal mostrando `dotnet new mvc -n GestBibliotecaApp` y `dotnet run` ejecutados correctamente.

3.4 Estructura de carpetas generada

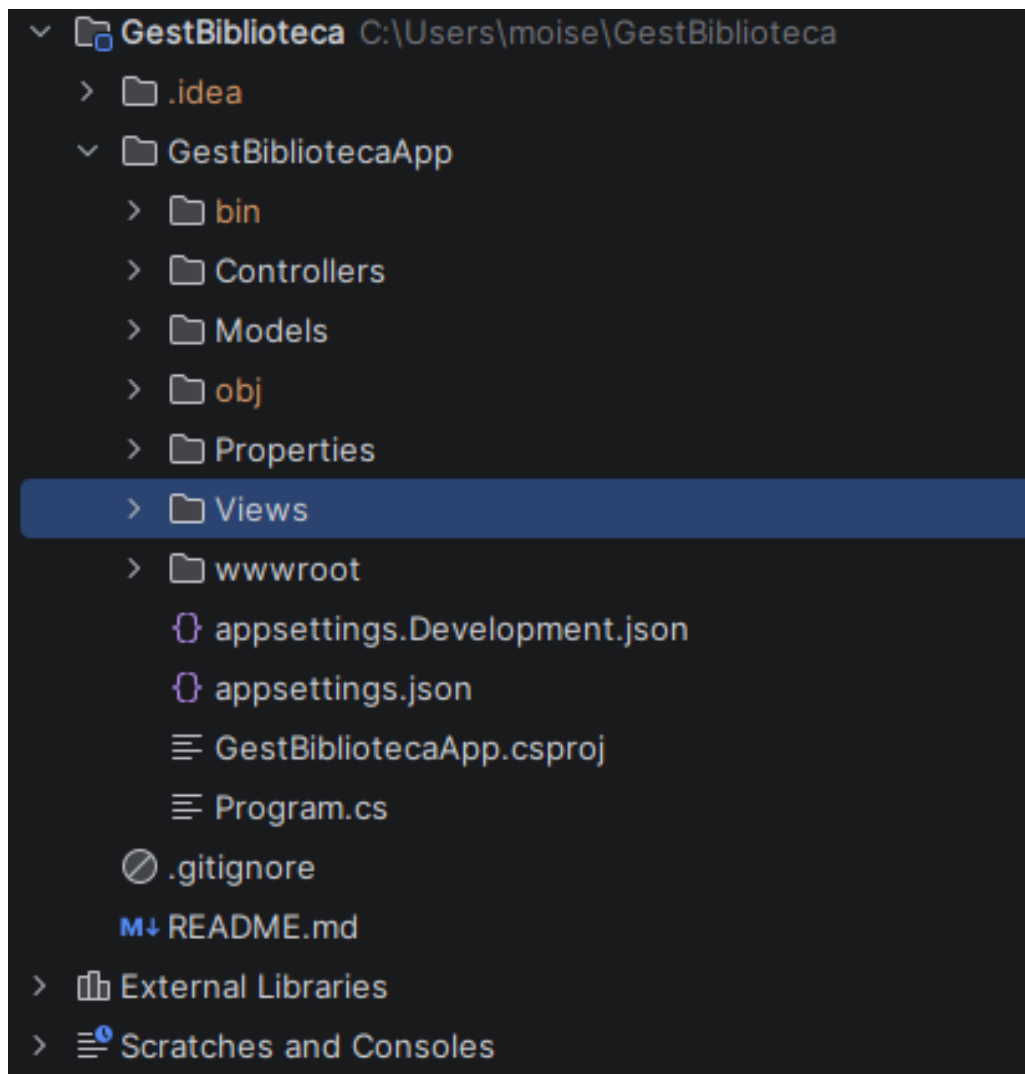


Figure 2: Estructura de carpetas del proyecto generado visible en IntelliJ IDEA.

4 Paso 2 — Modelo Prestamo y PrestamoController

4.1 Código del modelo

```
using System.ComponentModel.DataAnnotations;

namespace GestBibliotecaApp.Models
{
    public class Prestamo
    {
        public int Id { get; set; }

        [Required(ErrorMessage = "El nombre del estudiante es obligatorio")]
        [StringLength(100, ErrorMessage = "Maximo 100 caracteres")]
        [Display(Name = "Estudiante")]
        public string? Estudiante { get; set; }

        [Required(ErrorMessage = "El titulo del libro es obligatorio")]
        [StringLength(150, ErrorMessage = "Maximo 150 caracteres")]
        [Display(Name = "Libro")]
    }
}
```

```

public string? Libro { get; set; }

[Required(ErrorMessage = "La fecha de prestamo es obligatoria")]
[DataType(DataType.Date)]
[Display(Name = "Fecha de Prestamo")]
public DateTime FechaPrestamo { get; set; }

[Required(ErrorMessage = "La fecha de devolucion es obligatoria")]
[DataType(DataType.Date)]
[Display(Name = "Fecha de Devolucion Estimada")]
public DateTime FechaDevolucion { get; set; }

[Required(ErrorMessage = "El estado es obligatorio")]
[RegularExpression("Activo|Devuelto|Vencido|Renovado",
    ErrorMessage = "Use: Activo, Devuelto, Vencido o Renovado")]
[Display(Name = "Estado")]
public string? Estado { get; set; }
}
}

```

4.2 Archivo Prestamo.cs en el editor

Se creo el archivo Prestamo.cs dentro de la carpeta Models con las Data Annotations correspondientes a cada campo del modelo.

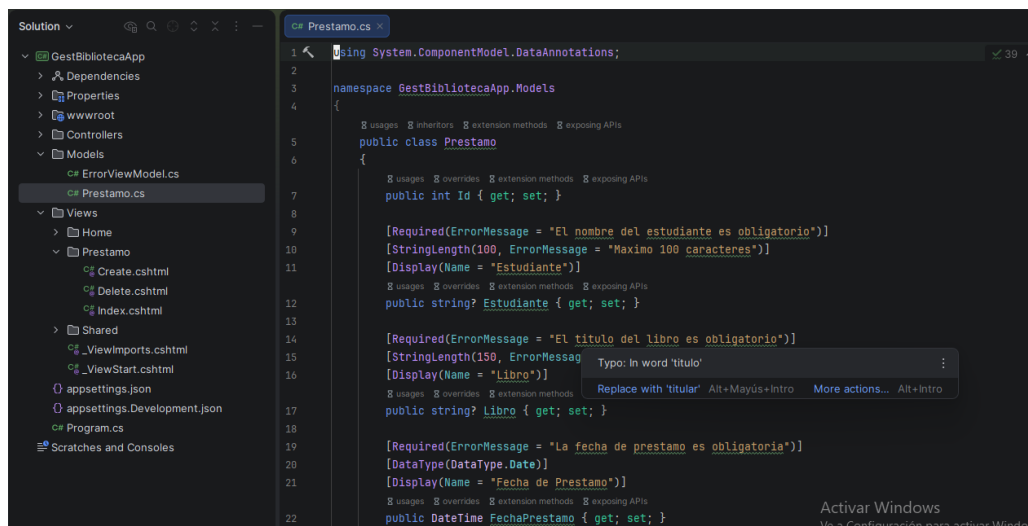


Figure 3: Archivo Prestamo.cs abierto en IntelliJ IDEA.

4.3 Codigo del controlador (Index y Create)

```

// GestBiblioteca - Modulo de Prestamos
// Extension: Delete con confirmacion POST y proteccion CSRF
using Microsoft.AspNetCore.Mvc;
using GestBibliotecaApp.Models;

namespace GestBibliotecaApp.Controllers
{
    public class PrestamoController : Controller
    {
        private static List<Prestamo> _prestamos = new List<Prestamo>();

        // GET: /Prestamo
        public IActionResult Index()

```

```
{
    return View(_prestamos);
}

// GET: /Prestamo/Create
public IActionResult Create()
{
    return View();
}

// POST: /Prestamo/Create
[HttpPost]
[ValidateAntiForgeryToken]
public IActionResult Create(Prestamo prestamo)
{
    if (ModelState.IsValid)
    {
        prestamo.Id = _prestamos.Count + 1;
        _prestamos.Add(prestamo);
        return RedirectToAction("Index");
    }
    return View(prestamo);
}

// GET: /Prestamo/Delete/5 (solo muestra confirmacion, nunca borra)
public IActionResult Delete(int id)
{
    var prestamo = _prestamos.FirstOrDefault(p => p.Id == id);
    if (prestamo == null)
    {
        return NotFound();
    }
    return View(prestamo);
}

// POST: /Prestamo/Delete/5 (borra real, protegido con CSRF)
[HttpPost, ActionName("Delete")]
[ValidateAntiForgeryToken]
public IActionResult DeleteConfirmed(int id)
{
    var prestamo = _prestamos.FirstOrDefault(p => p.Id == id);
    if (prestamo != null)
    {
        _prestamos.Remove(prestamo);
    }
    return RedirectToAction("Index");
}
}
```

4.4 Archivo PrestamoController.cs en el editor

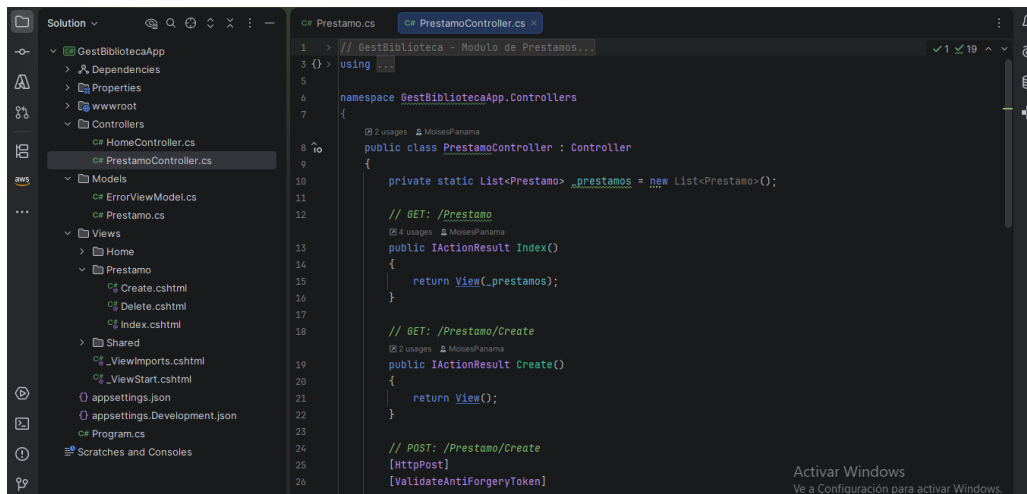


Figure 4: Archivo PrestamoController.cs con las acciones Index y Create en IntelliJ IDEA.

5 Paso 3 — Vistas Razor

5.1 Código de la vista Index

```
@* Vista Index - Lista de Prestamos *@
@model IEnumerable<GestBibliotecaApp.Models.Prestamo>

@{
    ViewData["Title"] = "Prestamos";
}

<div class="container mt-4">
    <h2 class="mb-3">Gestion de Prestamos</h2>

    <a asp-action="Create" class="btn btn-primary mb-3">+ Registrar Prestamo</a>

    @if (!Model.Any())
    {
        <div class="alert alert-info">No hay prestamos registrados aun.</div>
    }
    else
    {
        <table class="table table-bordered table-hover">
            <thead class="table-dark">
                <tr>
                    <th>#</th>
                    <th>Estudiante</th>
                    <th>Libro</th>
                    <th>F. Prestamo</th>
                    <th>F. Devolucion</th>
                    <th>Estado</th>
                    <th>Acciones</th>
                </tr>
            </thead>
            <tbody>
                @foreach (var p in Model)
                {
                    <tr>
                        <td>@p.Id</td>
```



```

        <td>@p.Estudiante</td>
        <td>@p.Libro</td>
        <td>@p.FechaPrestamo.ToShortDateString()</td>
        <td>@p.FechaDevolucion.ToShortDateString()</td>
        <td>
            <span class="badge @(p.Estado == "Activo" ? "bg-success" :
                p.Estado == "Vencido" ? "bg-danger" :
                p.Estado == "Renovado" ? "bg-warning text-dark" :
                "bg-secondary")">
                @p.Estado
            </span>
        </td>
        <td>
            <a asp-action="Delete" asp-route-id="@p.Id"
                class="btn btn-danger btn-sm">Eliminar</a>
        </td>
    </tr>
}
</tbody>
</table>
}
</div>

```

5.2 Vista Index en el navegador

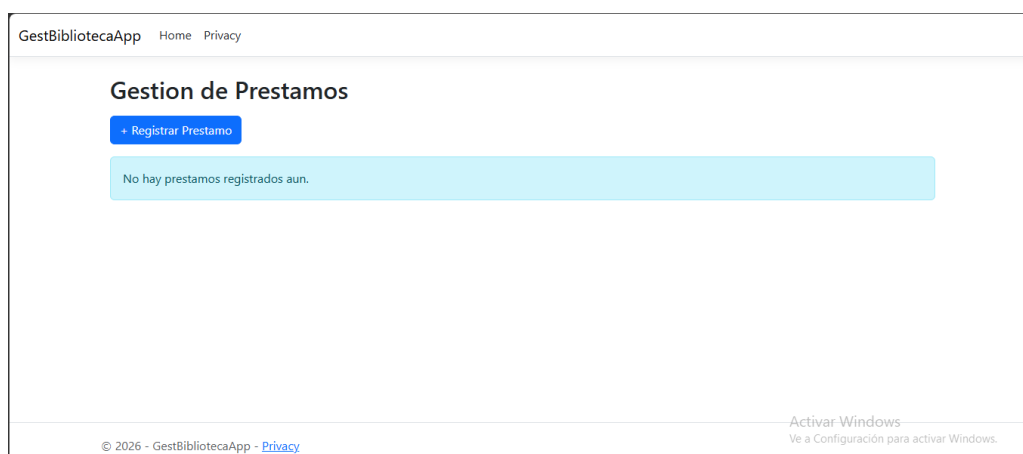


Figure 5: Vista Index en el navegador mostrando la tabla de prestamos.

5.3 Codigo de la vista Create

```

@* Vista Create - Registrar Prestamo - Validacion activada *@
@model GestBibliotecaApp.Models.Prestamo

@{
    ViewData["Title"] = "Registrar Prestamo";
}

<div class="container mt-4">
    <h2 class="mb-3">Registrar Prestamo</h2>

    <form asp-action="Create" method="post">
        @Html.AntiForgeryToken()

        <div class="mb-3">
            <label asp-for="Estudiante" class="form-label"></label>

```

```

        <input asp-for="Estudiante" class="form-control"
            placeholder="Nombre completo del estudiante" />
        <span asp-validation-for="Estudiante" class="text-danger"></span>
    </div>

    <div class="mb-3">
        <label asp-for="Libro" class="form-label"></label>
        <input asp-for="Libro" class="form-control"
            placeholder="Titulo del libro" />
        <span asp-validation-for="Libro" class="text-danger"></span>
    </div>

    <div class="mb-3">
        <label asp-for="FechaPrestamo" class="form-label"></label>
        <input asp-for="FechaPrestamo" class="form-control" type="date" />
        <span asp-validation-for="FechaPrestamo" class="text-danger"></span>
    </div>

    <div class="mb-3">
        <label asp-for="FechaDevolucion" class="form-label"></label>
        <input asp-for="FechaDevolucion" class="form-control" type="date" />
        <span asp-validation-for="FechaDevolucion" class="text-danger"></span>
    </div>

    <div class="mb-3">
        <label asp-for="Estado" class="form-label"></label>
        <select asp-for="Estado" class="form-control">
            <option value="">-- Seleccione un estado --</option>
            <option value="Activo">Activo</option>
            <option value="Devuelto">Devuelto</option>
            <option value="Vencido">Vencido</option>
            <option value="Renovado">Renovado</option>
        </select>
        <span asp-validation-for="Estado" class="text-danger"></span>
    </div>

    <button type="submit" class="btn btn-success">Guardar</button>
    <a asp-action="Index" class="btn btn-secondary">Cancelar</a>
</form>
</div>

@section Scripts {
    @{ await Html.RenderPartialAsync("_ValidationScriptsPartial"); }
}

```

5.4 Formulario Create en el navegador

El formulario incluye `@@Html.AntiForgeryToken()` para protegerse contra ataques CSRF. Al renderizar `_ValidationScriptsPartial` se habilita la **validacion cliente** (jQuery Unobtrusive Validation).

Figure 6: Formulario *Registrar Prestamo* en el navegador.

6 Paso 4 — Accion Delete con confirmacion, POST y CSRF

6.1Codigo del controlador (Delete)

```
// GET: /Prestamo/Delete/5 (solo muestra confirmacion, nunca borra)
public IActionResult Delete(int id)
{
    var prestamo = _prestamos.FirstOrDefault(p => p.Id == id);
    if (prestamo == null)
    {
        return NotFound();
    }
    return View(prestamo);
}

// POST: /Prestamo/Delete/5 (borra real, protegido con CSRF)
[HttpPost, ActionName("Delete")]
[ValidateAntiForgeryToken]
public IActionResult DeleteConfirmed(int id)
{
    var prestamo = _prestamos.FirstOrDefault(p => p.Id == id);
    if (prestamo != null)
    {
        _prestamos.Remove(prestamo);
    }
    return RedirectToAction("Index");
}
```

Se implemento un `Delete` GET que solo muestra una vista de confirmacion (nunca borra datos en una peticion GET), y un `Delete` POST (`DeleteConfirmed`) protegido con `[ValidateAntiForgeryToken]` que ejecuta la eliminacion real.

```
// GET: /Prestamo/Delete/5 (solo muestra confirmacion, nunca borra)
[HttpGet]
public IActionResult Delete(int id)
{
    var prestamo = _prestamos.FirstOrDefault(p : Prestamo => p.Id == id);
    if (prestamo == null)
    {
        return NotFound();
    }
    return View(prestamo);
}

// POST: /Prestamo/Delete/5 (borra real, protegido con CSRF)
[HttpPost, ActionName("Delete")]
[ValidateAntiForgeryToken]
public IActionResult DeleteConfirmed(int id)
{
    var prestamo = _prestamos.FirstOrDefault(p : Prestamo => p.Id == id);
    if (prestamo != null)
    {
        _prestamos.Remove(prestamo);
    }
}
```

Figure 7: Acciones Delete (GET) y DeleteConfirmed (POST + CSRF) en el controlador.

6.2 Vista de confirmacion en el navegador

GestBibliotecaApp Home Privacy

Eliminar Prestamo

Atencion: Esta accion no se puede deshacer. Confirma que deseas eliminar este registro.

Estudiante	Moisés Antonio Panamá Murillo
Libro	Harry Potter
Fecha de Prestamo	18/06/2026
Fecha de Devolucion	18/07/2026
Estado	Activo

[Confirmar eliminacion](#) [Cancelar](#)

© 2026 - GestBibliotecaApp - [Privacy](#)

Activar Windows
Ve a Configuración para activar Windows.

Figure 8: Vista de confirmacion antes de eliminar un prestamo.

7 Paso 5 — Verificacion de la validacion (server-side y cliente)

7.1 Errores de validacion en el formulario

Al enviar el formulario vacio o con datos invalidos, primero actua la **validacion cliente** (JavaScript) y, si esta se evita, la **validacion server-side**: el servidor evalua `ModelState.IsValid`; si es `false`, la accion `Create` regresa la misma vista mostrando los mensajes de error definidos en las Data Annotations, sin guardar el registro.

Registrar Prestamo

Estudiante

 El nombre del estudiante es obligatorio

Libro

 El título del libro es obligatorio

Fecha de Prestamo

 La fecha de prestamo es obligatoria

Fecha de Devolucion Estimada

 La fecha de devolucion es obligatoria

Estado

 El estado es obligatorio

Figure 9: Mensajes de error de validacion server-side.

7.2 Prestamo creado exitosamente

GestBibliotecaApp Home Privacy

Gestion de Prestamos

[+ Registrar Prestamo](#)

#	Estudiante	Libro	F. Prestamo	F. Devolucion	Estado	Acciones
1	Moisés Antonio Panamá Murillo	Harry Potter	18/06/2026	18/07/2026	Activo	Eliminar

© 2026 - GestBibliotecaApp - [Privacy](#)

Figure 10: Vista Index actualizada, mostrando el prestamo recién registrado.

8 Repositorio GitHub

8.1 Flujo de trabajo utilizado

1. **Version base en main:** proyecto, modelo Prestamo y acciones Index/Create.

```
git init
git add .
git commit -m "Paso 1: Proyecto MVC GestBiblioteca creado y probado"
git branch -M main
git remote add origin https://github.com/MoisesPanama/GestBiblioteca.git
git push -u origin main
```

2. **Rama de extension:** accion Delete con confirmacion, CSRF y validacion cliente.

```
git checkout -b feature/extension-delete-csrf
git add .
git commit -m "Paso 4: Accion Delete con confirmacion POST y CSRF"
git push origin feature/extension-delete-csrf
```

Luego: *Compare and pull request* → *Create pull request* → *Merge pull request* hacia **main**.

8.2 Historial de commits

#	Rama	Mensaje
1	main	Paso 1: Proyecto MVC GestBiblioteca creado y probado
2	main	Paso 2: Modelo Prestamo con Data Annotations y PrestamoController
3	main	Paso 3: Vistas Razor Index Create y Delete con AntiForgeryToken
4	feature/extension-delete-csrf	Paso 4: Accion Delete con confirmacion POST y CSRF
5	main	Paso 5: Validacion server-side verificada y proyecto finalizado

8.3 Enlace al repositorio

<https://github.com/MoisesPanama/GestBiblioteca>

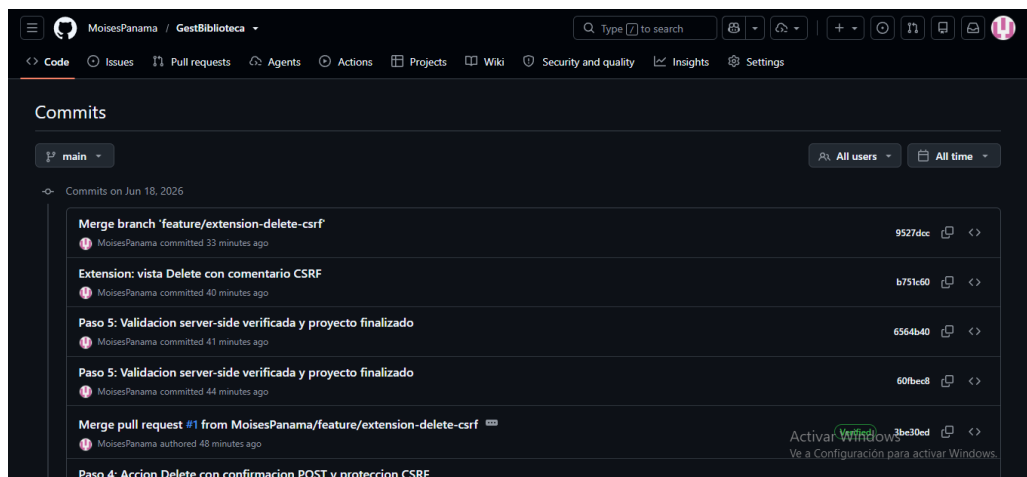


Figure 11: Repositorio en GitHub con el historial de commits y el Pull Request.

9 Extensión de la práctica — Tarea TA: Módulo de Productos

Como tarea de práctica fuera de clase (TA), se extendió el proyecto **GestBiblioteca** agregando un nuevo módulo de **Productos**, cumpliendo las directrices: tabla HTML con lista de productos, acción **Delete** con confirmación POST y CSRF, validación cliente y servidor, trabajo sobre la rama `feature/practica-aspnet`, y actualización del `README.md` con instrucciones de ejecución.

9.1 Paso 1 TA — Creación de la rama y el modelo Producto

```
git checkout main
git pull origin main
git checkout -b feature/practica-aspnet
```

```
using System.ComponentModel.DataAnnotations;

namespace GestBibliotecaApp.Models
{
    public class Producto
    {

```

```

public int Id { get; set; }

[Required(ErrorMessage = "El nombre del producto es obligatorio")]
[StringLength(100, ErrorMessage = "Mximo 100 caracteres")]
[Display(Name = "Nombre del producto")]
public string? Nombre { get; set; }

[Required(ErrorMessage = "La descripcin es obligatoria")]
[StringLength(250, ErrorMessage = "Mximo 250 caracteres")]
[Display(Name = "Descripcin")]
public string? Descripcion { get; set; }

[Required(ErrorMessage = "El precio es obligatorio")]
[Range(0.01, 10000, ErrorMessage = "El precio debe estar entre 0.01 y 10000")]
[DataType(DataType.Currency)]
[Display(Name = "Precio")]
public decimal Precio { get; set; }

[Required(ErrorMessage = "El stock es obligatorio")]
[Range(0, 5000, ErrorMessage = "El stock debe estar entre 0 y 5000")]
[Display(Name = "Stock disponible")]
public int Stock { get; set; }
}
}

```

Se creó la rama `feature/practica-aspnet` a partir de `main` y, dentro de ella, el archivo `Producto.cs` en la carpeta `Models`, con las Data Annotations correspondientes a cada campo.

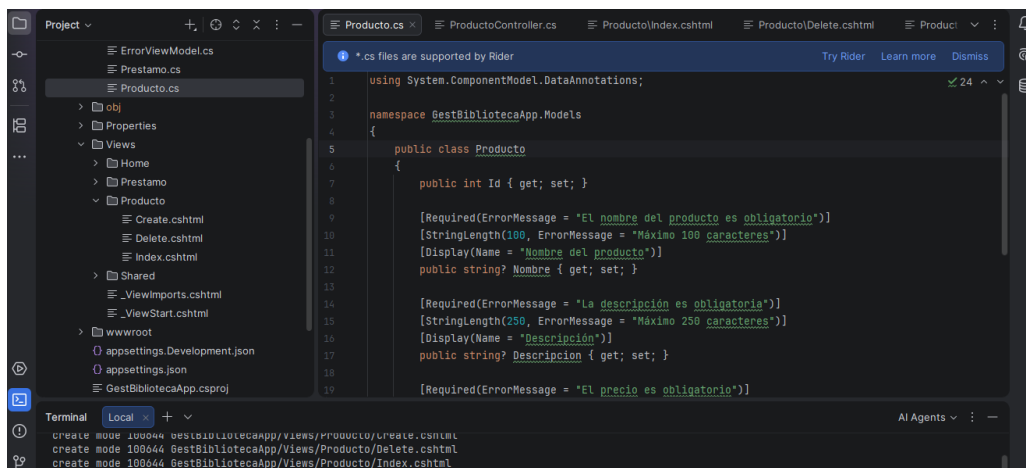


Figure 12: Archivo `Producto.cs` en IntelliJ IDEA, dentro de la rama `feature/practica-aspnet`.

9.2 Paso 2 TA — ProductoController

```

using Microsoft.AspNetCore.Mvc;
using GestBibliotecaApp.Models;

namespace GestBibliotecaApp.Controllers
{
    public class ProductoController : Controller
    {
        private static List<Producto> _productos = new List<Producto>();

        // GET: /Producto
        public IActionResult Index()
        {
            return View(_productos);
        }
    }
}

```

```
}

// GET: /Producto/Create
public IActionResult Create()
{
    return View();
}

// POST: /Producto/Create
[HttpPost]
[ValidateAntiForgeryToken]
public IActionResult Create(Producto producto)
{
    if (ModelState.IsValid)
    {
        producto.Id = _productos.Count + 1;
        _productos.Add(producto);
        return RedirectToAction("Index");
    }
    return View(producto);
}

// GET: /Producto/Delete/5 (solo confirmacion, nunca borra)
public IActionResult Delete(int id)
{
    var producto = _productos.FirstOrDefault(p => p.Id == id);
    if (producto == null)
    {
        return NotFound();
    }
    return View(producto);
}

// POST: /Producto/Delete/5 (borra real, protegido con CSRF)
[HttpPost, ActionName("Delete")]
[ValidateAntiForgeryToken]
public IActionResult DeleteConfirmed(int id)
{
    var producto = _productos.FirstOrDefault(p => p.Id == id);
    if (producto != null)
    {
        _productos.Remove(producto);
    }
    return RedirectToAction("Index");
}
}
```

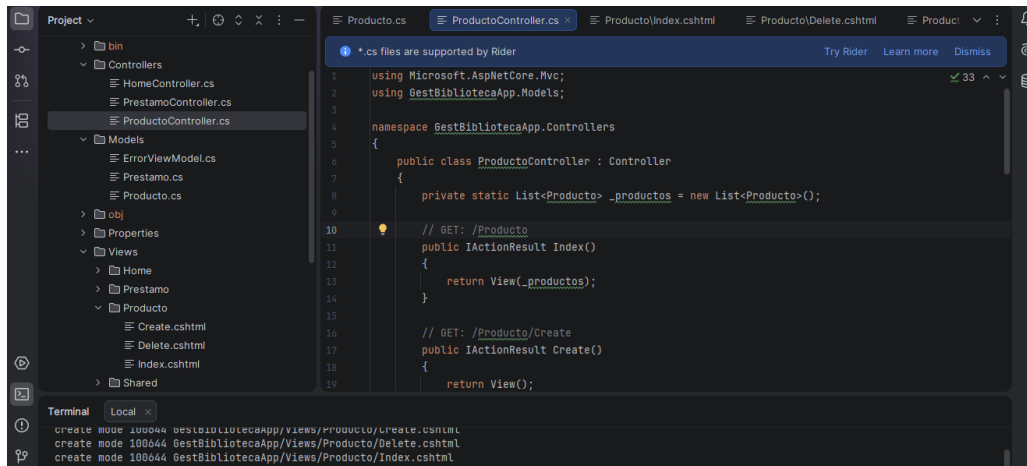



Figure 13: Archivo ProductController.cs con las acciones Index, Create y Delete (CSRF).

9.3 Paso 3 TA — Vistas Razor: tabla HTML, Create y Delete

```
@* Vista Index - tabla HTML con lista de productos (directriz 1) *@
@model IEnumerable<GestBibliotecaApp.Models.Producto>

@{
    ViewData["Title"] = "Productos";
}

<div class="container mt-4">
    <h2 class="mb-3">GestIn de Productos</h2>

    <a asp-action="Create" class="btn btn-primary mb-3">+ Agregar Producto</a>

    @if (!Model.Any())
    {
        <div class="alert alert-info">No hay productos registrados an.</div>
    }
    else
    {
        <table class="table table-bordered table-hover">
            <thead class="table-dark">
                <tr>
                    <th>#</th>
                    <th>Nombre</th>
                    <th>Descripcin</th>
                    <th>Precio</th>
                    <th>Stock</th>
                    <th>Acciones</th>
                </tr>
            </thead>
            <tbody>
                @foreach (var p in Model)
                {
                    <tr>
                        <td>@p.Id</td>
                        <td>@p.Nombre</td>
                        <td>@p.Descripcion</td>
                        <td>@p.Precio.ToString("C")</td>
                        <td>@p.Stock</td>
                        <td>
                            <a asp-action="Delete" asp-route-id="@p.Id"
                                class="btn btn-danger btn-sm">Eliminar</a>
                        </td>
                    </tr>
                }
            </tbody>
        </table>
    }
}
```

```

        </tr>
    }
    </tbody>
</table>
}
</div>

```

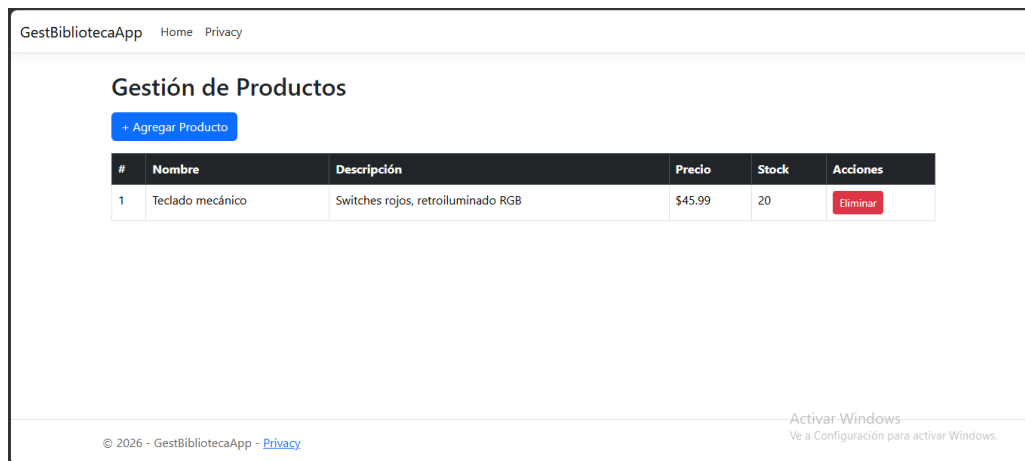


Figure 14: Vista Index en el navegador, mostrando la tabla HTML con la lista de productos.

```

@* Vista Create - validacion cliente y servidor (directriz 3) *@
@model GestBibliotecaApp.Models.Producto

@{
    ViewData["Title"] = "Agregar Producto";
}

<div class="container mt-4">
    <h2 class="mb-3">Agregar Producto</h2>

    <form asp-action="Create" method="post">
        @Html.AntiForgeryToken()

        <div class="mb-3">
            <label asp-for="Nombre" class="form-label"></label>
            <input asp-for="Nombre" class="form-control" placeholder="Ej: Teclado mecánico" />
            <span asp-validation-for="Nombre" class="text-danger"></span>
        </div>

        <div class="mb-3">
            <label asp-for="Descripcion" class="form-label"></label>
            <input asp-for="Descripcion" class="form-control" placeholder="Breve descripción" />
            <span asp-validation-for="Descripcion" class="text-danger"></span>
        </div>

        <div class="mb-3">
            <label asp-for="Precio" class="form-label"></label>
            <input asp-for="Precio" class="form-control" />
            <span asp-validation-for="Precio" class="text-danger"></span>
        </div>

        <div class="mb-3">
            <label asp-for="Stock" class="form-label"></label>
            <input asp-for="Stock" class="form-control" />
            <span asp-validation-for="Stock" class="text-danger"></span>
        </div>
    </form>

```

```

        <button type="submit" class="btn btn-success">Guardar</button>
        <a asp-action="Index" class="btn btn-secondary">Cancelar</a>
    </form>
</div>

@section Scripts {
    @{ await Html.RenderPartialAsync("_ValidationScriptsPartial"); }
}

```

Figure 15: Formulario *Agregar Producto* en el navegador.

```

@* Vista Delete - confirmacion antes de eliminar (directriz 2) *@
@model GestBibliotecaApp.Models.Producto

@{
    ViewData["Title"] = "Eliminar Producto";
}

<div class="container mt-4">
    <h2 class="mb-3">Eliminar Producto</h2>

    <div class="alert alert-warning">
        Est seguro de que desea eliminar el siguiente producto?
    </div>

    <dl class="row">
        <dt class="col-sm-2">Nombre</dt>
        <dd class="col-sm-10">@Model.Nombre</dd>

        <dt class="col-sm-2">Descripcin</dt>
        <dd class="col-sm-10">@Model.Descripcion</dd>

        <dt class="col-sm-2">Precio</dt>
        <dd class="col-sm-10">@Model.Precio.ToString("C")</dd>

        <dt class="col-sm-2">Stock</dt>
        <dd class="col-sm-10">@Model.Stock</dd>
    </dl>

    <form asp-action="Delete" method="post">
        @Html.AntiForgeryToken()
        <input type="hidden" asp-for="Id" />
        <button type="submit" class="btn btn-danger">Confirmar eliminacin</button>
        <a asp-action="Index" class="btn btn-secondary">Cancelar</a>
    </form>

```

```
</form>
</div>
```

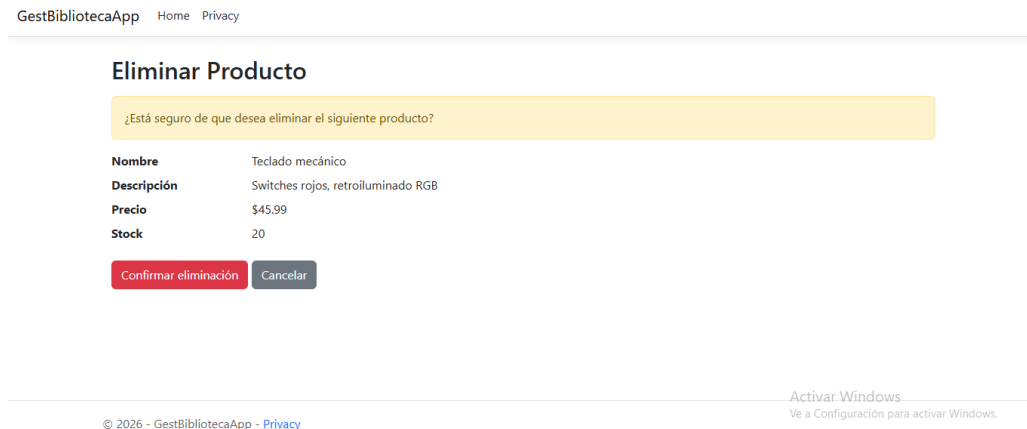


Figure 16: Vista de confirmación antes de eliminar un producto (directriz 2: confirmación POST + CSRF).

9.4 Paso 4 TA — Verificación de validación cliente y servidor

Se comprobaron dos niveles de validación, tal como exige la directriz 3:

1. **Validación cliente:** al intentar enviar el formulario vacío, jQuery Unobtrusive Validation muestra los errores inmediatamente, sin recargar la página.
2. **Validación servidor:** al desactivar JavaScript y reenviar el formulario, el servidor evalúa `ModelState.IsValid` y devuelve los mismos mensajes de error, confirmando que la validación también ocurre del lado del servidor y no depende únicamente del navegador.

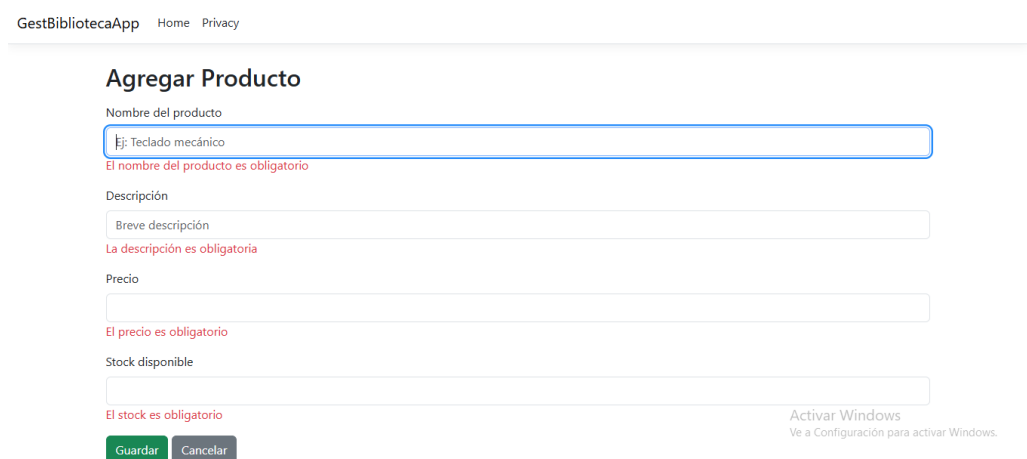


Figure 17: Mensajes de error de validación (cliente y servidor) en el formulario de Producto.

Adicionalmente, se detectó y corrigió un problema de formato: por defecto, la cultura del sistema mostraba los precios con coma decimal y símbolo €. Se corrigió fijando la cultura del proyecto en `Program.cs`:

```
using System.Globalization;

var builder = WebApplication.CreateBuilder(args);
```

```
builder.Services.AddControllersWithViews();

var cultureInfo = new CultureInfo("en-US");
CultureInfo.DefaultThreadCurrentCulture = cultureInfo;
CultureInfo.DefaultThreadCurrentUICulture = cultureInfo;

var app = builder.Build();
```

Gestión de Productos

+ Agregar Producto

#	Nombre	Descripción	Precio	Stock	Acciones
1	Teclado mecánico	Switches rojos, retroiluminado RGB	\$45.99	20	Eliminar

Figure 18: Producto creado exitosamente, con el precio mostrado correctamente en formato \$45.99.

9.5 Paso 5 TA — README actualizado

Se actualizó el archivo `README.md` del repositorio, agregando la descripción del nuevo módulo de Productos, sus rutas de acceso, y las instrucciones de ejecución para ambos módulos (Préstamos y Productos).

9.6 Flujo de Git/GitHub — Rama feature/practica-aspnet

```
git checkout -b feature/practica-aspnet
git add Models/Producto.cs
git commit -m "Paso 1 TA: Modelo Producto con Data Annotations"
git push -u origin feature/practica-aspnet

git add Controllers/ProductoController.cs
git commit -m "Paso 2 TA: ProductoController con Index, Create y Delete (CSRF)"
git push

git add Views/Producto/
git commit -m "Paso 3 TA: Vistas Razor Index (tabla HTML), Create y Delete"
git push

git add Program.cs
git commit -m "Fix: cultura en-US para formato correcto de precio"
git push

git add README.md
git commit -m "Paso 5 TA: README actualizado con modulo de Productos"
git push
```

Posteriormente se abrió un Pull Request en GitHub desde `feature/practica-aspnet` hacia `main`, se revisó el diff y se realizó el *merge*.

Historial de commits — Tarea TA

#	Rama	Mensaje
1	feature/practica-aspnet	Paso 1 TA: Modelo Producto con Data Annotations
2	feature/practica-aspnet	Paso 2 TA: ProductoController con Index, Create y Delete (CSRF)
3	feature/practica-aspnet	Paso 3 TA: Vistas Razor Index (tabla HTML), Create y Delete
4	feature/practica-aspnet	Fix: cultura en-US para formato correcto de precio
5	feature/practica-aspnet	Paso 5 TA: README actualizado con modulo de Productos

9.7 Pull Request y enlace al repositorio

<https://github.com/MoisesPanama/GestBiblioteca/pulls>

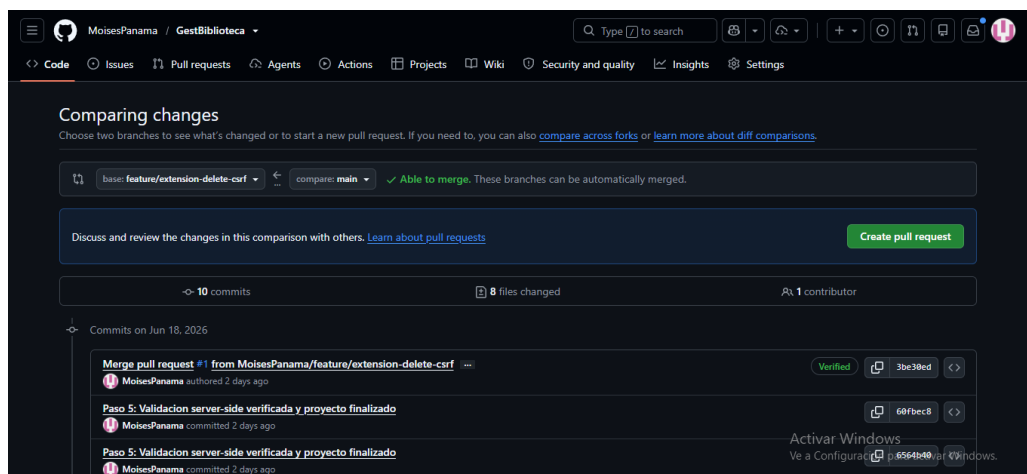


Figure 19: Pull Request de la rama `feature/practica-aspnet` hacia `main`.



Figure 20: Repositorio en GitHub mostrando el historial de commits de la rama `feature/practica-aspnet` ya integrada.

10 Conclusiones

El desarrollo de **GestBiblioteca** permitió comprender de forma práctica el patrón MVC en ASP.NET Core, tanto en la práctica en clase (GA) como en su extensión como tarea fuera de clase (TA). El **modelo** centraliza las reglas de validación mediante Data Annotations, el **controlador** orquesta el flujo entre las peticiones HTTP y los datos, y la **vista** Razor se encarga de la presentación con Tag Helpers. La acción **Delete** con confirmación, implementada tanto en el módulo de Préstamos como en el de Productos, reforzó la importancia de no ejecutar operaciones destructivas en peticiones GET. El uso combinado de **AntiForgeryToken** con validación cliente y server-side demostró cómo ASP.NET Core MVC protege la integridad de los datos en múltiples capas. Finalmente, trabajar con dos ramas de extensión (**feature/extension-delete-csrf** y **feature/practica-aspnet**) y sus respectivos Pull Requests permitió practicar un flujo de control de versiones cercano al de un proyecto de equipo real, manteniendo **main** siempre estable mientras se desarrollaban nuevas funcionalidades en paralelo.